



郑州大学
Zhengzhou University

第四单元 哈希函数与消息认证码

郑州大学网络空间安全学院

王志华

王志华

第4单元 哈希函数与消息认证码

提纲

CONTENTS

4.1 哈希函数概述

4.2 MD算法

4.3 SHA算法

4.4 MAC

4.5 SM3

4.6 其他哈希函数

王华

4.4.1 MAC概述

●消息认证的需求

- ① **完整性**: 接收方验证其收到的消息是否遭到**第三方有意或无意的篡改**——Hash函数
- ② **真实性**: 接收方验证其收到的消息是否是由**声称的主体**发来的 ——消息认证码

王卫华



1、MAC的定义

- 消息的**真实性认证**依赖消息认证码（Message Authenticaion Code），简称MAC。

$$\square \text{MAC} = C_{\text{K}}(m)$$

2022/4/26



1、MAC的定义

●消息认证码是指消息被一密钥控制的公开函数作用后产生的、用作认证符的、固定长度的数值，也称为密码校验和。

➤需要通信双方Alice和Bob共享一密钥K

➤假设Alice欲发送给Bob的消息是M，则

- ① Alice首先计算 $MAC = C_K(M)$ ，其中 $C_K(\cdot)$ 是密钥控制的公开函数，
- ② 然后Alice向Bob发送 $M || MAC$ ，
- ③ Bob收到后，做与Alice相同的计算，求得一新MAC，并与收到的MAC做比较。

2022/4/26



2、MAC的作用:

◆ 功能

- ◆ 数据完整性
- ◆ 数据源认证

对比HASH函数的作用!!!

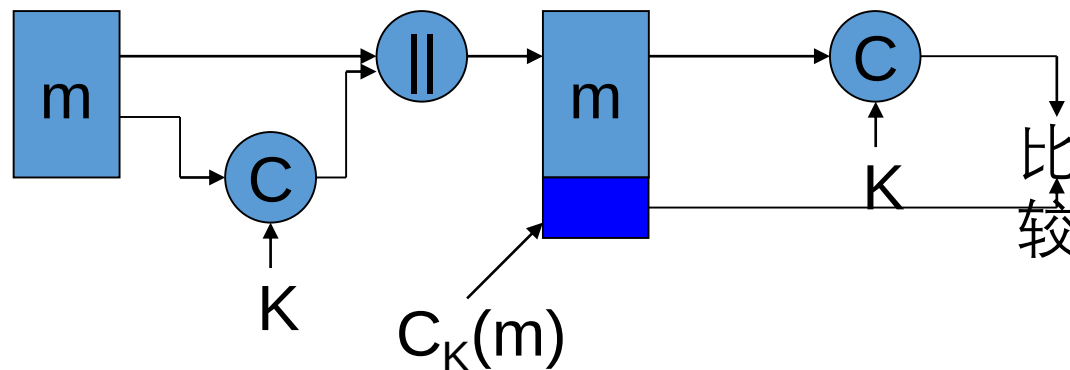
2022/4/26



2、MAC的作用:

■完整性认证和数据源认证

- 对明文消息和发送者身份进行认证--K



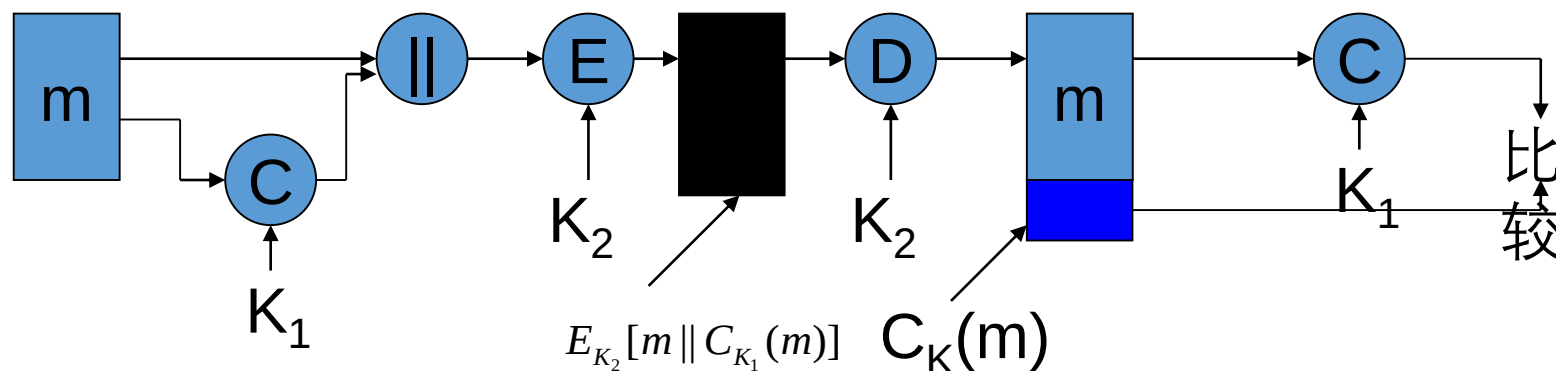
2022/4/26



2、MAC的作用：

■完整性认证性、数据源认证和保密性

- 对**明文消息**和数据源身份进行认证—K1
- 对明文消息和认证码进行加密—K2

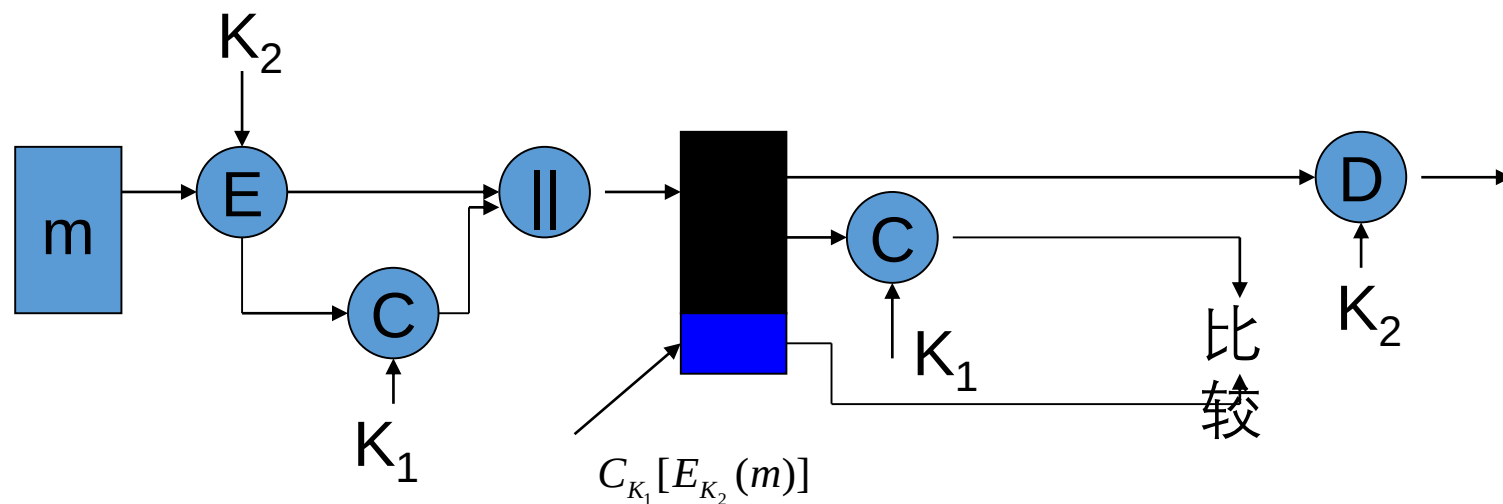


2022/4/26

2、MAC的作用:

■完整性认证性、数据源认证和保密性

- 对**密文消息**和数据源身份进行认证—K1
- 对明文消息进行加密—K2



2022/4/26



3、MAC的使用方式

●MAC用来提供**消息认证**的基本使用方式，共有以下

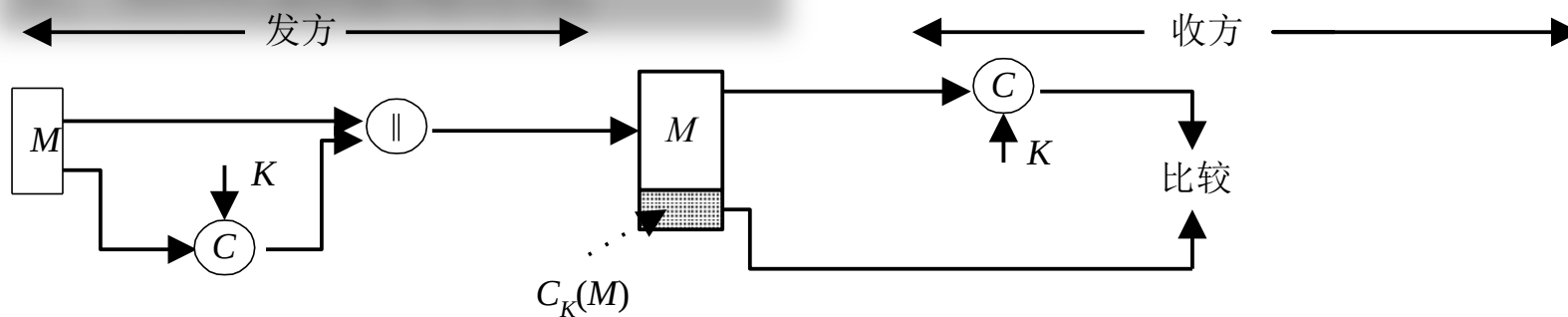
3种：第一部分密码学概论

- 重点思考每种使用方法都能**满足哪些安全需求**！
- 重点思考每种使用方法都能**抵抗哪些安全攻击**！
- 重点思考每种使用方法都能**提供哪些安全服务**！
- 重点思考每种使用方法都能**适用哪些应用场景**！

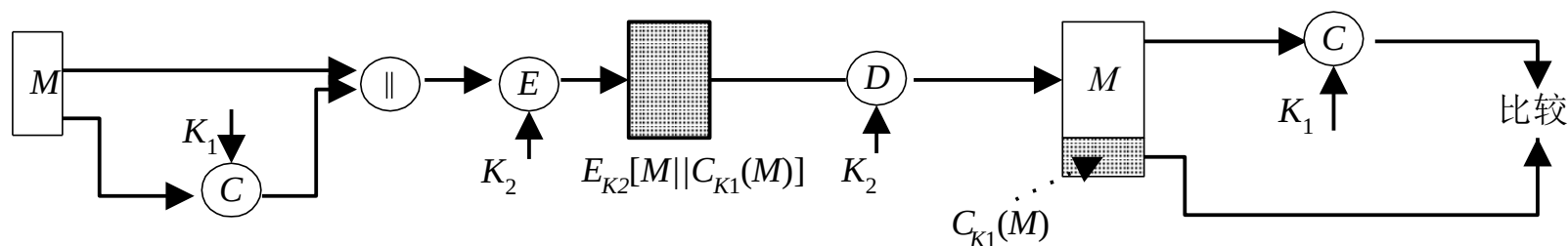
王卫华



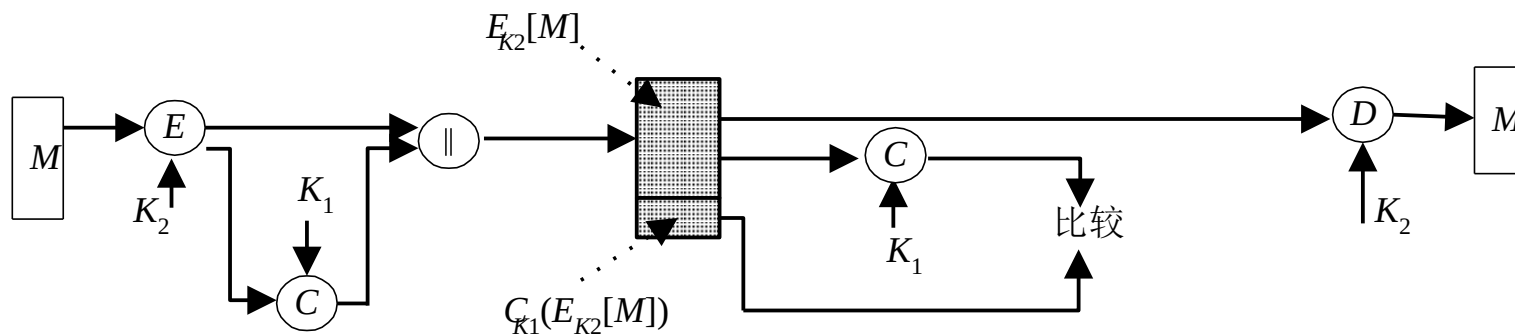
3、MAC的使用方式



(a) 消息认证



(b) 认证性和保密性：对明文认证



(c) 认证性和保密性：对密文认证



3、MAC的使用方式：消息认证

- 如果仅收发双方知道 K ，且B计算得到的 MAC 与接收到的 MAC 一致，则这一系统就实现了以下功能：
 - ① 接收方相信发方发来的消息**未被篡改**，这是因为攻击者不知道密钥，所以不能够在篡改消息后相应地篡改 MAC ，而如果仅篡改消息，则接收方计算的新 MAC 将与收到的 MAC 不同。
 - ② 接收方相信发方**不是冒充的**，这是因为除收发双方外再无其他人知道密钥，因此其他人不可能对自己发送的消息计算出正确的 MAC 。
- 上述过程中，由于消息本身在发送过程中是**明文形式**，所以这一过程**只提供认证性而未提供保密性**。
- MAC 函数与加密算法类似，不同之处为 **MAC 函数不必是可逆的**，因此与加密算法相比更不易被攻破。

王立华



3、MAC的使用方式：认证和加密

- 为提供**保密性**，可在MAC函数以后（如图6-1（b））或以前（如图6-1（c））进行**一次加密**，而且**加密密钥也需被收发双方共享**。
 - 在图6-1（b）中，M与MAC链接后再被整体加密，因此对**明文消息进行认证**
 - 在图6-1（c）中，M先被加密再与MAC链接后发送，因此对**密文消息进行认证**
- 通常更希望**直接对明文进行认证**，因此图6-1（b）所示的使用方式更为常用。

2022/4/26



4、MAC的安全性:

● 加密算法的密钥:

- 使用**加密算法**加密消息时，其安全性一般取决于**加密密钥的长度**。
- 由于加密算法是公开的，所以敌手一般使用**穷举密钥搜索攻击**以测试所有可能的密钥。
- 如果加密密钥长为k比特，则**穷举密钥搜索攻击**平均将进行 2^{k-1} 个测试。
- 特别地，对**惟密文攻击**来说，敌手如果知道密文C，则将对所有可能的密钥值 $P_i = D_{K_i}(C)$ 执行解密运算，直到得到有意义的明文。

王卫华



4、MAC的安全性：密钥穷举攻击

● 消息认证函数的密钥：

- 产生MAC的函数一般都是**多到一的映射**。
- 如果**产生n比特长的MAC**，则函数的取值范围即为 2^n 个可能的MAC；函数输入的可能的消息个数 $N \gg 2^n$ 。
- 如果函数所用的密钥为k比特，则**可能的密钥个数为** 2^k 。
- 如果系统**不考虑保密性**，即敌手能获取明文消息和相应的MAC，在这种情况下考虑敌手使用**穷举密钥搜索攻击**以获取产生MAC的函数所使用的**密钥**。

王卫华



4、MAC的安全性: $k > n$ 密钥分析

- 假定 $k > n$ ，且敌手已得到 M_1 和 MAC_1 ，其中 $MAC_1 = C_{K_1}(M_1)$ ，敌手对所有可能的密钥值 K_i 求 $MAC_i = C_{K_i}(M_1)$ ，直到找到某个 K_i 得 $MAC_i = MAC_1$ 。
- 由于不同的密钥个数为 2^k ，因此将产生 2^k 个MAC，但其中仅有 2^n 个不同，由于 $2^k > 2^n$ ，所以**有很多密钥（平均有 $2^k/2^n = 2^{k-n}$ 个）都可产生出正确的 MAC_1** ，而敌手无法知道进行通信的两个用户用的是哪一个密钥，还必须按以下方式**重复上述攻击**：

2022/4/26



4、MAC的安全性: $k > n$ 密钥分析

- ① 第1轮: 已知 M_1 、 MAC_1 ，其中 $MAC_1 = C_K(M_1)$ 。对所有 2^k 个可能的密钥计算 $MAC_i = C_{K_i}(M_1)$ ，得 2^{k-n} 个可能的密钥。
- ② 第2轮: 已知 M_2 、 MAC_2 ，其中 $MAC_2 = C_K(M_2)$ 。对上一轮得到的 2^{k-n} 个可能的密钥计算 $MAC_i = C_{K_i}(M_2)$ ，得 2^{k-2n} 个可能的密钥。
- 如此下去，如果 $k = \alpha n$ （假定 $k > n$ ），则上述攻击方式**平均需要 α 轮**。例如，密钥长为80比特，MAC长为32比特，则第1轮将产生大约 2^{48} 个可能密钥，第2轮将产生 2^{16} 个可能的密钥，**第3轮即可找出正确的密钥**。

王华



4、MAC的安全性： $k < n$ 密钥分析

- 如果密钥长度小于MAC的长度（ $k < n$ ）：
 - 则第1轮就有可能找出正确的密钥
 - 也有可能找出多个可能的密钥
 - 如果是后者，则仍需执行第2轮搜索。

王华



4、MAC的安全性：密钥穷举攻击

- 所以对消息认证码的密钥穷举搜索攻击比对使用相同长度密钥的加密算法的密钥穷举搜索攻击的代价还要大。

2022/4/26



4、MAC的安全性：消息伪造攻击

- 然而有些攻击法却**不需要寻找产生MAC所使用的密钥**, 比如**消息伪造攻击**:
 - 针对消息完整性, 对手最主要的攻击目标就是能够成功伪造消息对应的MAC!

王华



4、MAC的安全性：消息伪造攻击

- 例如，设 $M = (X_1 \parallel X_2 \parallel \dots \parallel X_m)$ 是由64比特长的分组 $X_i (i=1, \dots, m)$ 链接得到的，其**消息认证码**由以下方式得到：

$$\Delta(M) = X_1 \oplus X_2 \oplus \dots \oplus X_m$$

$$C_K(M) = E_K[\Delta(M)]$$

- 其中 $\oplus$ 表示异或运算，加密算法是**电码本模式的DES**。
- 因此，**密钥长为56比特，MAC长为64比特**。
- 如果敌手得到 $M \parallel C_K(M)$ ，则以**密钥穷举搜索攻击**寻找K将需做 2^{56} 次加密。

2022/4/26



4、MAC的安全性：消息伪造攻击

- 例如，设 $M = (X_1 \parallel X_2 \parallel \cdots \parallel X_m)$ 是由64比特长的分组 $X_i (i=1, \dots, m)$ 链接得到的，其**消息认证码**由以下方式得到：

$$\Delta(M) = X_1 \oplus X_2 \oplus \cdots \oplus X_m$$

$$C_K(M) = E_K[\Delta(M)]$$

- 然而敌手可用**以下方式攻击系统**：将 X_1 到 X_{m-1} 分别用自己选取的 Y_1 到 Y_{m-1} 替换，求出 $Y_m = Y_1 \oplus Y_2 \oplus \cdots \oplus Y_{m-1} \oplus \Delta(M)$ （敌手由 M 易得 $\Delta(M)$ ），并用 Y_m 替换 X_m 。
- 因此**敌手可成功伪造一新消息** $M' = Y_1 \oplus \cdots \oplus Y_m$ ，**且 M' 的MAC与原消息 M 的MAC相同。**

2022/4/26



5、MAC的设计要求:

- 考虑到MAC所存在的**以上消息伪造攻击**，在假定**敌手知道函数 $C_K(M)$ 但不知道密钥 K 的情形下**，可得**MAC应满足以下要求**:
 - ① 如果敌手得到 M 和 $C_K(M)$ ，则构造一满足 $C_K(M')=C_K(M)$ 的新消息 M' 在计算上是不可行的；
 - ② $C_K(M)$ 在以下意义下是均匀分布的：随机选取两个消息 M 、 M' ， $P[C_K(M)=C_K(M')]=2^{-n}$ ，其中 n 为MAC的长。
 - ③ 若 M' 是 M 的某个变换，即 $M'=f(M)$ ，例如 f 为插入一个或多个比特，那么 $P[C_K(M)=C_K(M')]=2^{-n}$ 。

2022/4/26



5、MAC的设计要求:

- 考虑到MAC所存在的**以上消息伪造攻击**，在假定**敌手知道函数 $C_k(M)$** 但不知道密钥**K**的情形下，可得**MAC应满足以下要求**:
 - ① 第一个要求是针对消息伪造攻击，即敌手**不需要找出密钥K**而**伪造一个与截获的MAC相匹配的新消息在计算上是不可行的**。
 - ② 第二个要求是说敌手**如果截获一个MAC**，则**伪造一个相匹配的消息的概率为最小**。
 - ③ 第三个要求是说**函数 $C_k(M)$** 不应在消息的某个**部分或某些比特弱于其它部分或其它比特**。否则敌手获得M和MAC后就有**可能修改M中弱的部分**，从而**伪造出一个与原MAC相匹配的新消息**。

王卫华



6、MAC与Hash的区别：

- ◆ 与Hash函数（不带密钥）的关系
 - ◆ 密钥
 - ◆ 主要安全威胁

- ① Hash函数是防碰撞
- ② MAC是防伪造

2022/4/26



6、MAC的分类:

◆ 构造方法

- ◆ 通过不带密钥的Hash函数构造：HMAC
- ◆ 通过对称密码算法：CBC-MAC

2022/4/26

4.4.2 CBC-MAC

- CBC-MAC是最为广泛使用的消息认证算法中的一个。
- CBC-MAC已做为FIPS Publication (FIPS PUB 113)，并被ANSI作为X9.17标准。
- CBC-MAC实际上就是对消息使用CBC模式进行加密，取密文的最后一块作为认证码。
- CBC-MAC可以取DES或AES作为加密的分组密码，称为基于DES的CBC-MAC，或基于AES的CBC-MAC。

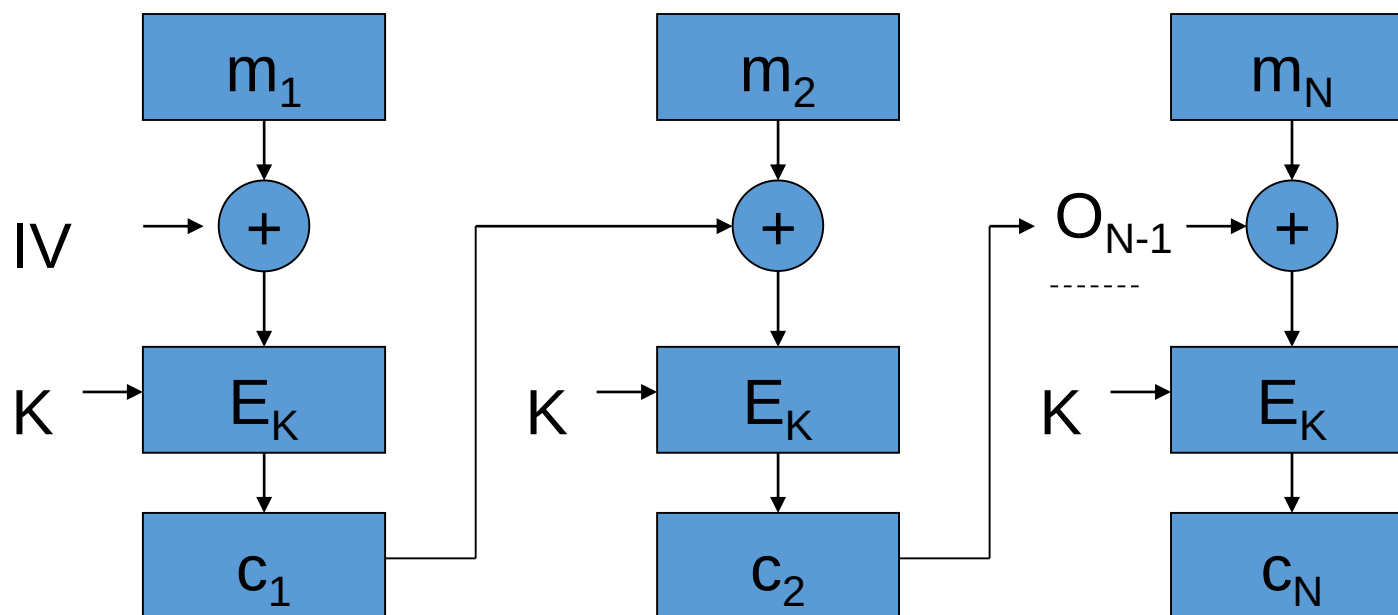
2022/4/26

2022/4/26



1、CBC-MAC算法概述

- CBC-MAC本质上就是基于CBC模式的分组加密算法，其初始向量取为零向量。



2022/4/26



2、CBC-MAC算法基本思想

- 以DES算法作为分组加密算法，需被认证的数据（消息、记录、文件或程序）被分为**64比特长的分组** $D_1, D_2, \dots, D_N$ ，其中如果最后一个分组不够64比特，可在其**右边填充一些0**，然后按以下过程计算**数据认证码**：

$$O_1 = E_K(D_1)$$

$$O_2 = E_K(D_2 \oplus O_1)$$

$$O_3 = E_K(D_3 \oplus O_2)$$

$\vdots$

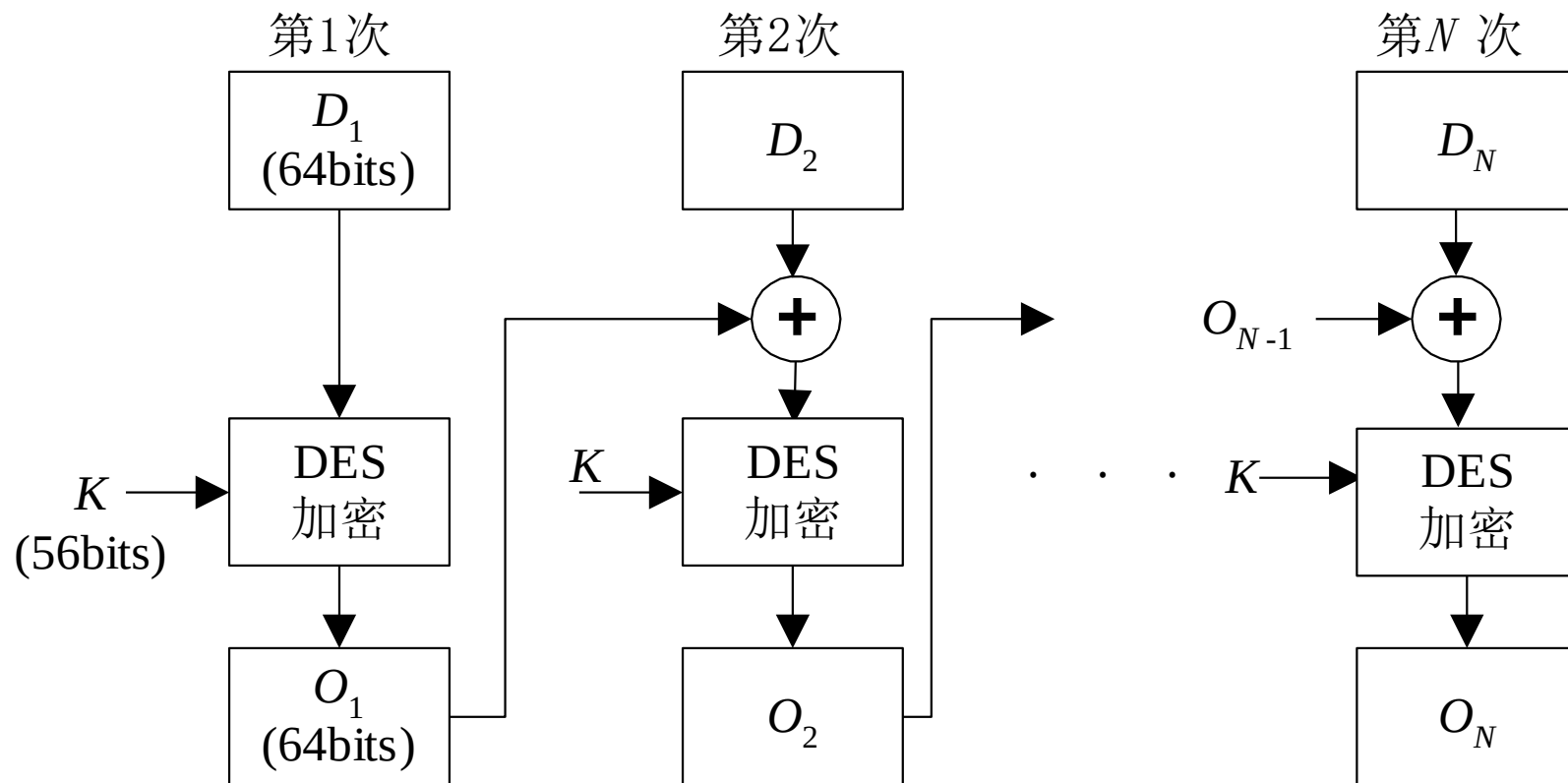
$$O_N = E_K(D_N \oplus O_{N-1})$$

- 其中**E为DES加密算法**，**K为密钥**。
- 数据认证码或者**取为** O_N 或者**取为** O_N 的最左M个比特，其中 $16 \leq M \leq 64$

王华



2、CBC-MAC算法基本思想



2022/4/26



3、CBC-MAC算法实现过程

$CBC(x, k)$

令 $x = x_1 || x_2 || \dots || x_n$

$IV \leftarrow 00\dots 0$

$y_0 \leftarrow IV$

for $i \leftarrow 1$ to n

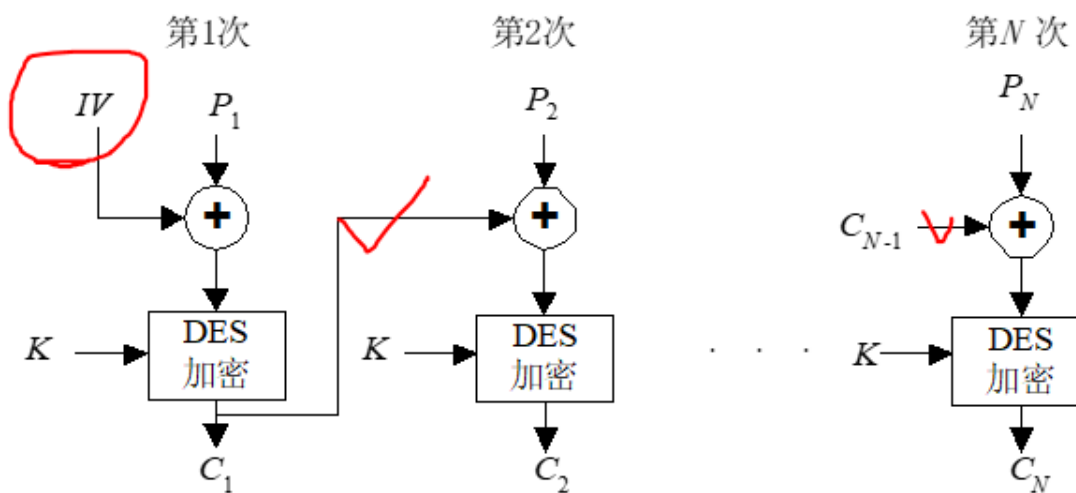
do $y_i \leftarrow e_k(y_{i-1} \oplus x_i)$

return(y_n)

2024

4、CBC-MAC算法总结

- ◆ 加密算法具有混乱特性，当基本加密算法满足合适的安全性质时，CBC-MAC是安全的。
- ◆ 与CBC加密模式的区别
 - ◆ IV
 - ◆ 中间结果



(a) 加密

4.4 MAC



4.4.3 HMAC

- CBC-MAC算法反映了传统上构造MAC最为普遍使用的方法，即**基于分组密码的构造方法**。
- 但近年来研究构造MAC的兴趣已转移到**基于密码哈希函数的构造方法**。
- HMAC (Hash-based Message Authentication Code) 是**密钥相关的哈希运算消息认证码**。
- HMAC由H. Krawczyk, M. Bellare, R. Canetti于1996年提出的一种**基于Hash函数和密钥进行消息认证**的方法，并于1997年作为RFC2104被公布。
- HMAC可以与任何**迭代散列函数**捆绑使用。

王卫华



1、HMAC概述

●使用HMAC的原因:

- ① 密码哈希函数(如MD5, SHA)的**软件实现快于**分组密码(如DES)的软件实现。
- ② 密码哈希函数的**库代码来源广泛**。
- ③ 密码哈希函数**没有出口限制**, 而分组密码即使用于MAC也有出口限制。

王立华



1、HMAC概述

- 哈希函数并不是为用于MAC而设计的，由于哈希函数不使用密钥，因此**哈希函数不能直接用于MAC**。

- ◆ 2002年3月被提议作为FIPS标准
- ◆ 通过（不带密钥）的Hash函数构造MAC

2022/4/26



2、HMAC设计目标

- RFC2104列举了HMAC的以下设计目标：
 - ① 可不经修改而**使用现有的哈希函数**，特别是那些易于软件实现的、源代码可方便获取且免费使用的哈希函数。
 - ② 其中镶嵌的**哈希函数可易于替换**为更快或更安全的哈希函数。
 - ③ 保持镶嵌的**哈希函数的最初性能**，不因用于HMAC而使其性能降低。
 - ④ 以**简单方式使用和处理密钥**。
 - ⑤ 在对镶嵌的哈希函数合理假设的基础上，**易于分析HMAC用于认证时的密码强度**。

王卫华



2、HMAC设计目标：分析

- 其中前两个目标是HMAC被公众普遍接受的主要原因，这两个目标是**将哈希函数当作一个黑盒使用**，这种方式有两个优点：
 - ① 第一个优点是哈希函数的实现可作为实现HMAC的**一个模块**，这样一来，HMAC代码中很大一块就可事先准备好，**无需修改**就可使用。
 - ② 第二个优点是如果HMAC要求使用更快或更安全的哈希函数，则只需**用新模块代替旧模块**，例如用实现SHA的模块代替MD5的模块。

王卫华



2、HMAC设计目标：分析

- 最后一条设计目标则是HMAC优于其他基于哈希函数的MAC的一个主要方面：
 - HMAC在其镶嵌的哈希函数具有合理密码强度的假设下，可证明是安全的。

王卫华



3、HMAC算法描述：参数

- HMAC算法的运行框图，其中：
 - H 为嵌入的哈希函数(如MD5, SHA),
 - M 为HMAC的输入消息(包括哈希函数所要求的填充位),
 - $Y_i (0 \leq i \leq L-1)$ 是 M 的第 i 个分组,
 - L 是 M 的分组数,
 - b 是一个分组中的比特数,
 - n 为由嵌入的哈希函数所产生的哈希值的长度,
 - k 为密钥, 如果密钥长度大于 b , 则将密钥输入到哈希函数中产生一个 n 比特长的密钥,
 - k^+ 是左边经填充0后的 k , k^+ 的长度为 b 比特,
 - $ipad$ 为 $\frac{b}{8}$ 个00110110,
 - $opad$ 为 $\frac{b}{8}$ 个01011010

吴志华



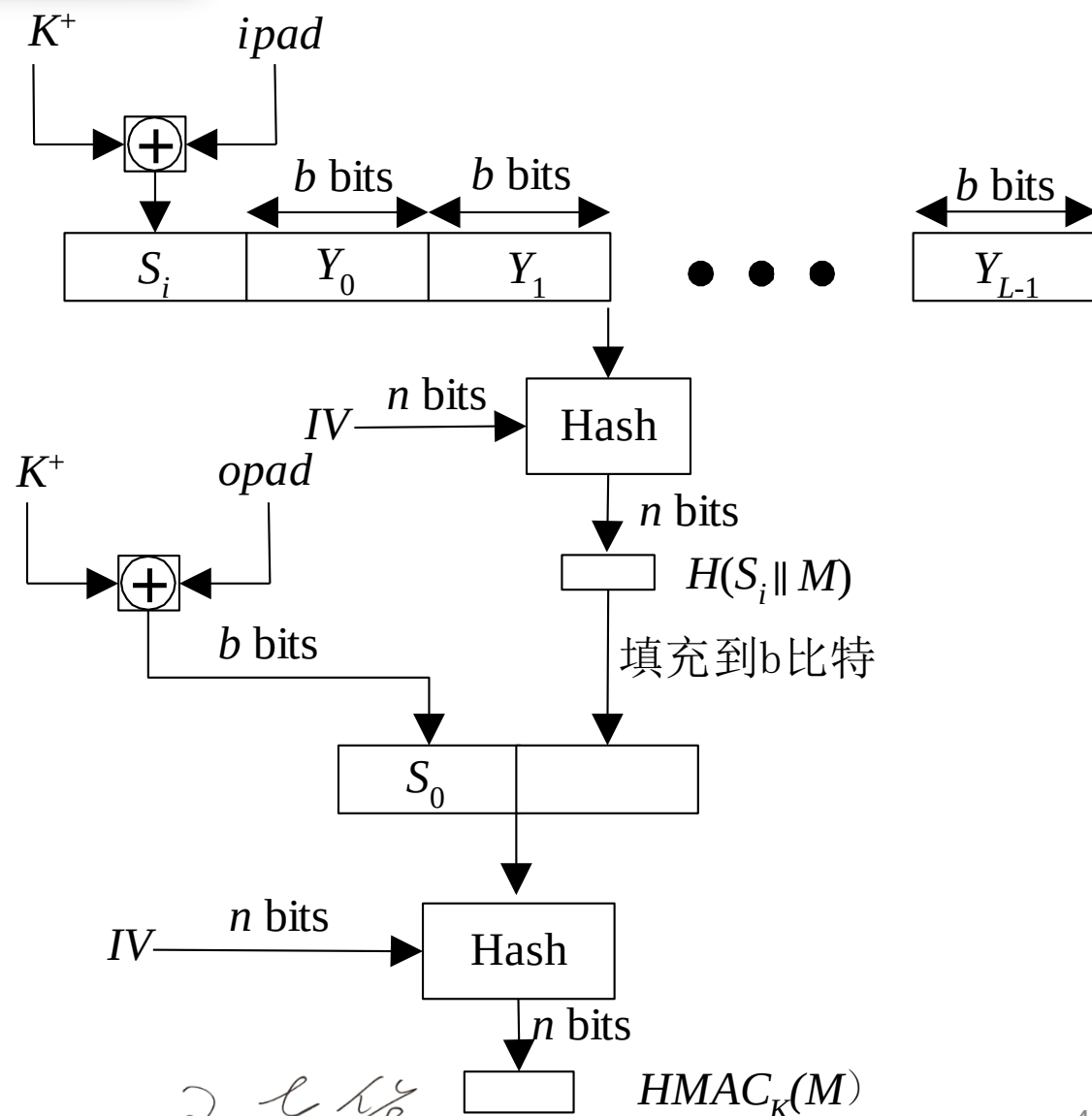
3、HMAC算法描述：图示

◆ 通过 (不带密钥) 的Hash函数构造MAC

$ipad=3636.....36$

$opad=5C5C.....5C$

$$HMAC_K(x) = SHA-1((K \oplus opad) || SHA-1((K \oplus ipad) || x))$$



2022/4/26



3、HMAC算法描述：过程

算法的输出可如下表达：

$$HMAC_k = H[(K^+ \oplus opad) \parallel H[(K^+ \oplus ipad) \parallel M]]$$

算法的运行过程可描述如下：

- (1) k 的左边填充0以产生一个 b 比特长的 k^+ (例如 k 的长为160比特, $b=512$, 则需填充44个零字节0x00)；
- (2) k^+ 与 $ipad$ 逐比特异或以产生 b 比特的分组 S_i ；
- (3) 将 M 链接到 s_i 后；
- (4) 将 H 作用于第3步产生的数据流；
- (5) k^+ 与 $opad$ 逐比特异或以产生 b 比特长的分组 ；
- (6) 将第4步得到的哈希值链接在 S_0 后；
- (7) 将 H 作用于第6步产生的数据流并输出最终结果。

2022/4/26



3、HMAC算法描述：分析

注意， k^+ 与 $ipad$ 逐比特异或和与 $opad$ 逐比特异或
其结果是将 k 中的一半比特取反，但两次取反的比特的
位置不同。而 s_i 和 s_0 通过哈希函数中压缩函数的处理，
则相当于以伪随机方式从 k 产生两个密钥。

$ipad$ 为 $b/8$ 个 00110110,

$opad$ 为 $b/8$ 个 01011010

吴志华



3、HMAC算法描述：预运算

在实现HMAC时，可预先求出下面两个量（看图6-13，虚线以左为预计算）：

$$f(IV, (K^+ \oplus ipad))$$

$$f(IV, (K^+ \oplus opad))$$

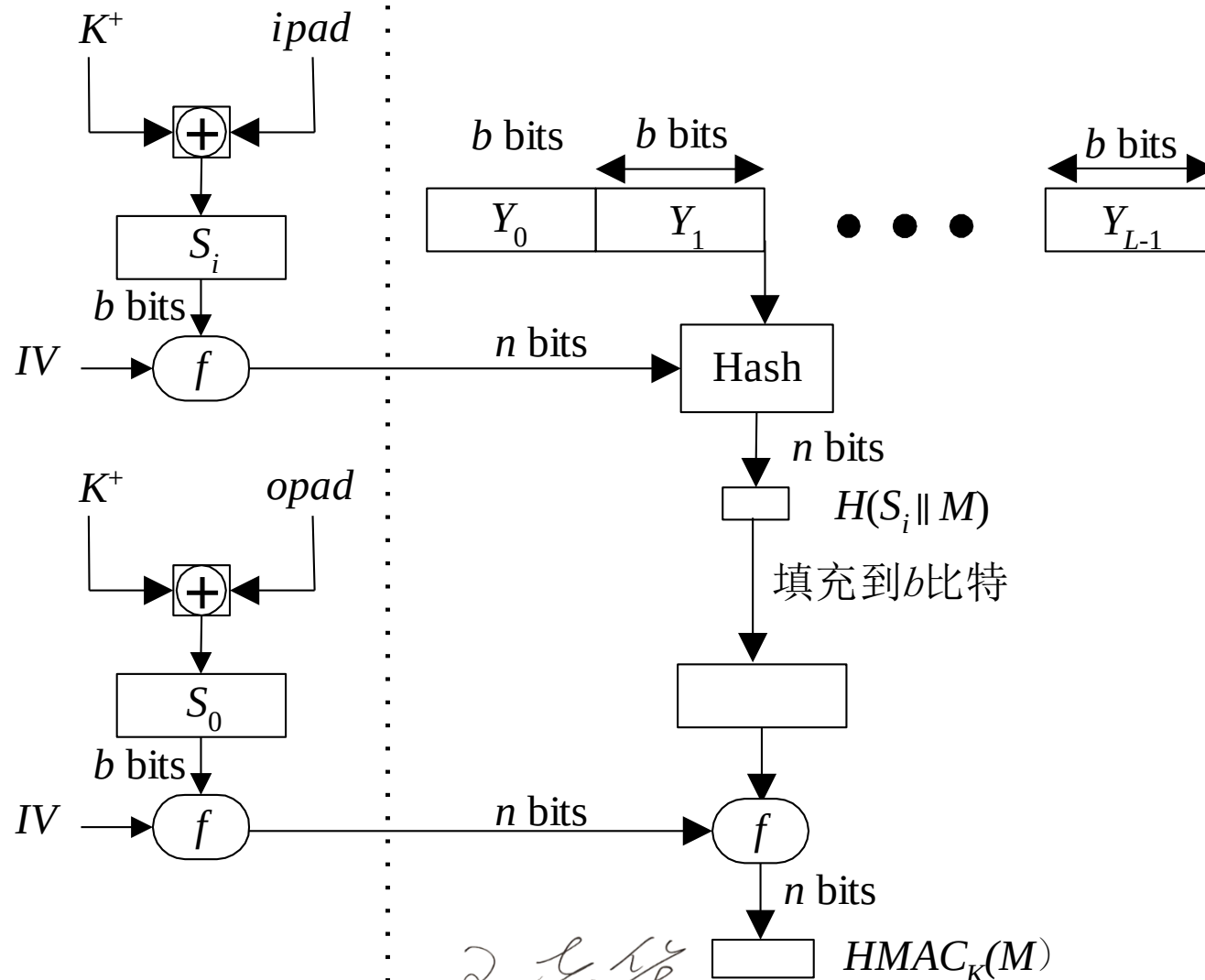
其中 $f(cv, block)$ 是哈希函数中的压缩函数，其输入是 n 比特的链接变量和 b 比特的分组，输出是 n 比特的链接变量。这两个量的预先计算只需在每次更改密钥时才被进行。事实上这两个预先计算的量用于作为哈希函数的初值 IV 。



王卫华



3、HMAC算法描述：预运算





4、HMAC的安全性

- 基于密码哈希函数构造的MAC的安全性**取决于镶嵌的哈希函数**的安全性。
- HMAC最具吸引人的地方是它的设计者已经证明了算法的强度和嵌入的哈希函数的强度之间的确切关系，证明了**对HMAC的攻击等价于对内嵌哈希函数的下述两种攻击之一**：
 - ① 攻击者能够计算压缩函数的一个输出，即使IV是随机的和秘密的。
 - ② 攻击者能够找出哈希函数的碰撞，即使IV是随机的和秘密的。

王卫华



4、HMAC的安全性

在第一种攻击中，我们可将压缩函数视为与哈希函数等价，而哈希函数的 n 比特长 IV 可视为HMAC的密钥。对这一哈希函数的攻击或者可通过对密钥的穷搜索来进行或者可通过第II类生日攻击来实施，通过对密钥的穷搜索攻击的复杂度为 $O(2^n)$ ，通过第II类生日攻击又可归结为上述第二种攻击。

第二种攻击指攻击者寻找具有相同哈希值的两个消息，因此就是第II类生日攻击。对哈希值长度为 n 的哈希函数来说，攻击的复杂度为 $O(2^{n/2})$ 。因此第二种攻击对MD5的攻击复杂度为 $O(2^n)$ ，就现在的技术来说，这种攻击是可行的。

2022/4/26



4、HMAC的安全性

但这是否意味着MD5不适合用于HMAC？回答是否定的原因如下：攻击者在攻击MD5时，可选择任何消息集合后脱线寻找碰撞。由于攻击者知道哈希算法和缺省的IV，因此能为自己产生的每个消息求出哈希值。然而，在攻击HMAC时，由于攻击者不知道密钥K，从而不能脱线产生消息和认证码对。所以攻击者必须得到HMAC在同一密钥下产生的一系列消息，并对得到的消息序列进行攻击。对长128比特的哈希值来说，需要得到用同一密钥产生的 2^{64} 个分组 (2^{73} 比特)。在1Gbps的链路上，需250000年，因此MD5完全适合于HMAC，而且就速度而言，快于SHA作为内嵌哈希函数的HMAC。

2022/4/26



5、HMAC总结

- ◆ 嵌套MAC的安全性
- ◆ 可替换其中的Hash函数

2022



6、MAC总结

- ◆ MAC的定义、功能和主要安全威胁
- ◆ MAC的构造方法
 - ◆ HMAC
 - ◆ CBC-MAC

2022/4/26

第4单元 哈希函数与消息认证码

提纲

CONTENTS

4.1 哈希函数概述

4.2 MD算法

4.3 SHA算法

4.4 MAC

4.5 SM3

4.6 其他哈希函数

王华



4.5.1 SM3: 中国商用密码

- SM3
 - 密码哈希算法;
 - 适用于商用密码应用中的**数字签名和验证, 消息认证码的生成与验证以及随机数的生成**, 可满足多种密码应用的安全需求。

王卫华



1、SM3概述

●SM3 密码哈希算法的输入数据长度为 ℓ 比特, $1 \leq \ell \leq 2^{64} - 1$, 输出哈希值长度为**256比特**。

- SM3密码哈希算法对**输入**长度小于2的64次方的比特消息, 经过填充和迭代压缩, **生成**长度为256比特的哈希值;
- SM3密码哈希算法使用了异或、模、模加、移位、与、或、非**运算**;
- SM3密码哈希算法由填充, 迭代过程, 消息扩展和压缩函数等**过程**所构成。

王卫华



2、SM3算法基础：参数

ABCDEFGH: 8个字寄存器或它们的值的串联

$B^{(i)}$: 第 i 个消息分组

CF: 压缩函数

FF_j : 布尔函数, 随 j 的变化取不同的表达式

GG_j : 布尔函数, 随 j 的变化取不同的表达式

IV: 初始值, 用于确定压缩函数寄存器的初态

P_0 : 压缩函数中的置换函数

P_1 : 消息扩展中的置换函数

T_j : 常量, 随 j 的变化取不同的值

m : 消息

m' : 填充后的消息

2022



2、SM3算法基础：运算

mod: 模运算

$\wedge$: 32比特与运算

$\vee$: 32比特或运算

$\oplus$: 32比特异或运算

$\neg$: 32比特非运算

$+$: $\text{mod}2^{32}$ 算术加运算

$\ll k$: 循环左移k比特

$\leftarrow$: 左向赋值运算符

王卫华



2、SM3算法基础：常量

初始值 $IV = 7380166F\ 4914B2B9$
 $172442D7DA8A0600$
 $A96F30BC163138AA$
 $E38DEE4DB0FB0E4E$

$$T_j = \begin{cases} 79CC4519, & 0 \leq j \leq 15 \\ 7A879D8A, & 16 \leq j \leq 63 \end{cases}$$

2022/4/26



2、SM3算法基础：函数

布尔函数：

$$FF_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z, & 0 \leq j \leq 15 \\ (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z), & 16 \leq j \leq 63 \end{cases}$$

$$GG_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z, & 0 \leq j \leq 15 \\ (X \wedge Y) \vee (\bar{X} \wedge Z), & 16 \leq j \leq 63 \end{cases}$$

式中 X, Y, Z 为32位字, $\wedge, \vee, -, \oplus$ 分别是逻辑与、逻辑或、逻辑非和逐比特异或运算

王华



2、SM3算法基础：函数

置换函数：

$$P_0(X) = X \oplus (X \lll 9) \oplus (X \lll 17);$$

$$P_1(X) = X \oplus (X \lll 15) \oplus (X \lll 23).$$

式中 X 为32位字，符号 $a \lll n$ 表示把 a 循环左移 n 位。

王华



3、SM3算法描述：概述

- 算法对数据首先进行**填充**，再进行**迭代压缩**后生成哈希值。

王华



3、SM3算法描述：第一步

(1) 填充并附加消息的长度

设消息 m 的长度为 ℓ 比特，这一步与MD5算法相同。

算法填充（如同MD5）

假设消息 m 的长度为 l 比特。首先将比特“1”添加到消息的末尾，再添加 k 个“0”， k 是满足

$$l + 1 + k \equiv 448 \pmod{512}$$

的最小的非负整数。然后再添加一个64位比特串，该比特串是 m' 的长度的二进制表示。填充后的消息的比特长度为512的倍数。

王卫华



3、SM3算法描述：第二步

(2) 迭代压缩

将填充后的消息 m' 按512比特进行分组得 $m' = B^{(0)}B^{(1)} \dots B^{(L-1)}$

对 m' 按下列方式迭代压缩：

FOR $i=0$ to $L-1$ $V^{(i+1)} = CF(V^{(i)}, B^{(i)})$

其中 CF 是压缩函数， $V^{(0)}$ 为 256 比特初始值 IV， $B^{(i)}$ 为填充后的消息分组，迭代压缩的结果为 $V^{(L)}$ ， $V^{(L)}$ 即为消息 m 的哈希值。

王卫华



3、SM3算法描述：第三步

(3) 消息扩展

在对消息分组 $B^{(i)}$ 进行迭代压缩之前，首先对其进行消息扩展，步骤如下：

① 消息分组 $B^{(i)}$ 划分为16个字 $W_0, W_1, \dots, W_{15}$ 。

② FOR $j=16$ to 67

$$W_j = P_1(W_{j-16} \oplus W_{j-9} \oplus (W_{j-3} \lll 15)) \oplus (W_{j-13} \lll 7) \oplus W_{j-6}$$

③ FOR $j=0$ to 63

$$W'_j = W_j \oplus W_{j+4}$$

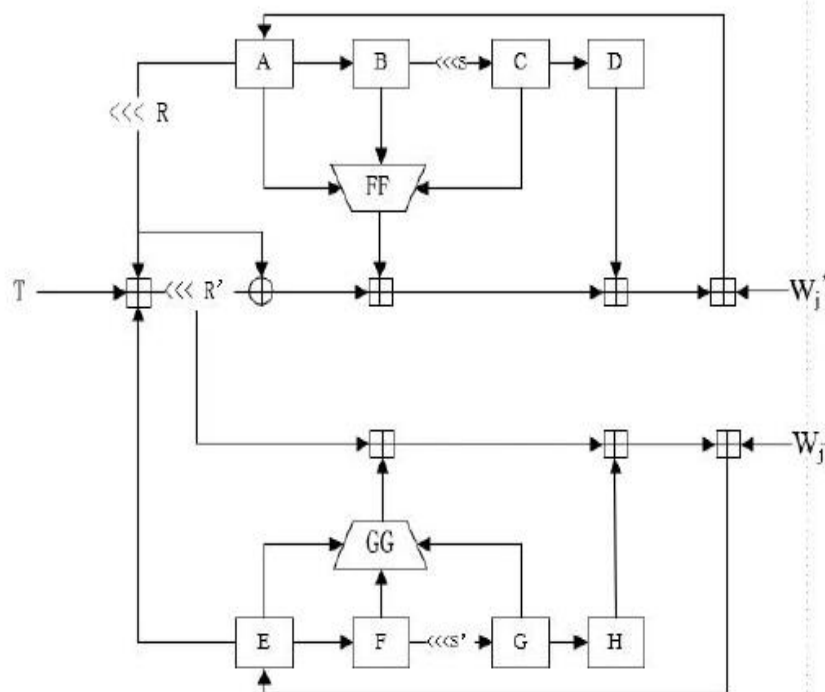
$B^{(i)}$ 经消息扩展后得到 $W_0, W_1, \dots, W_{67}, W'_0, W'_1, \dots, W'_{63}$ 。

2024

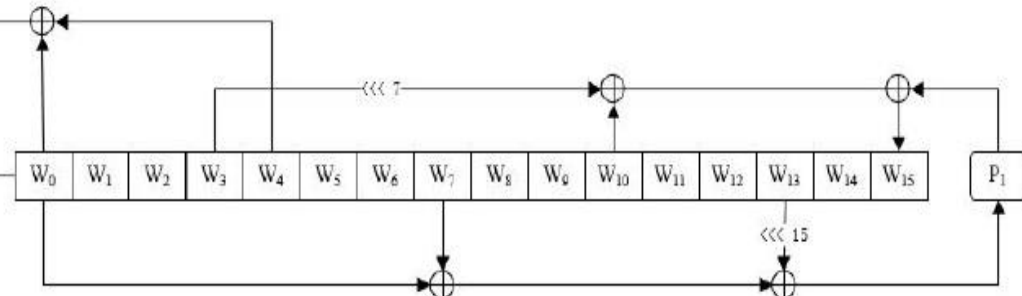


3、SM3算法描述：第三步

状态更新过程



消息扩散过程



王立华



3、SM3算法描述：第四步

(4) 压缩函数

设 A, B, C, D, E, F, G, H 为字寄存器, SS_1, SS_2, TT_1, TT_2 为中间变量, 压缩函数 $V^{(i+1)} = CF(V^{(i)}, B^{(i)})$ ($0 \leq i \leq n-1$) 的计算过程如下:

$$ABCDEFGH = V^{(i)}$$

FOR $j=0$ to 63

$$SS_1 \leftarrow ((A \lll 12) + E + (T_j \lll j)) \lll 7;$$

$$SS_2 = SS_1 \oplus (A \lll 12);$$

$$TT_1 = FF_j(A, B, C) + D + SS_2 + W'_j;$$

$$TT_2 = GG_j(E, F, G) + H + SS_1 + W_j;$$

2024



3、SM3算法描述：第四步

$$TT_2 = GG_j(E, F, G) + H + SS_1 + W_j;$$

$$D = C;$$

$$C = B \lll 9;$$

$$B = A;$$

$$A = TT_1;$$

$$H = G;$$

$$G = F \lll 19;$$

$$F = E;$$

$$E = P_0(TT_2)$$

ENDFOR

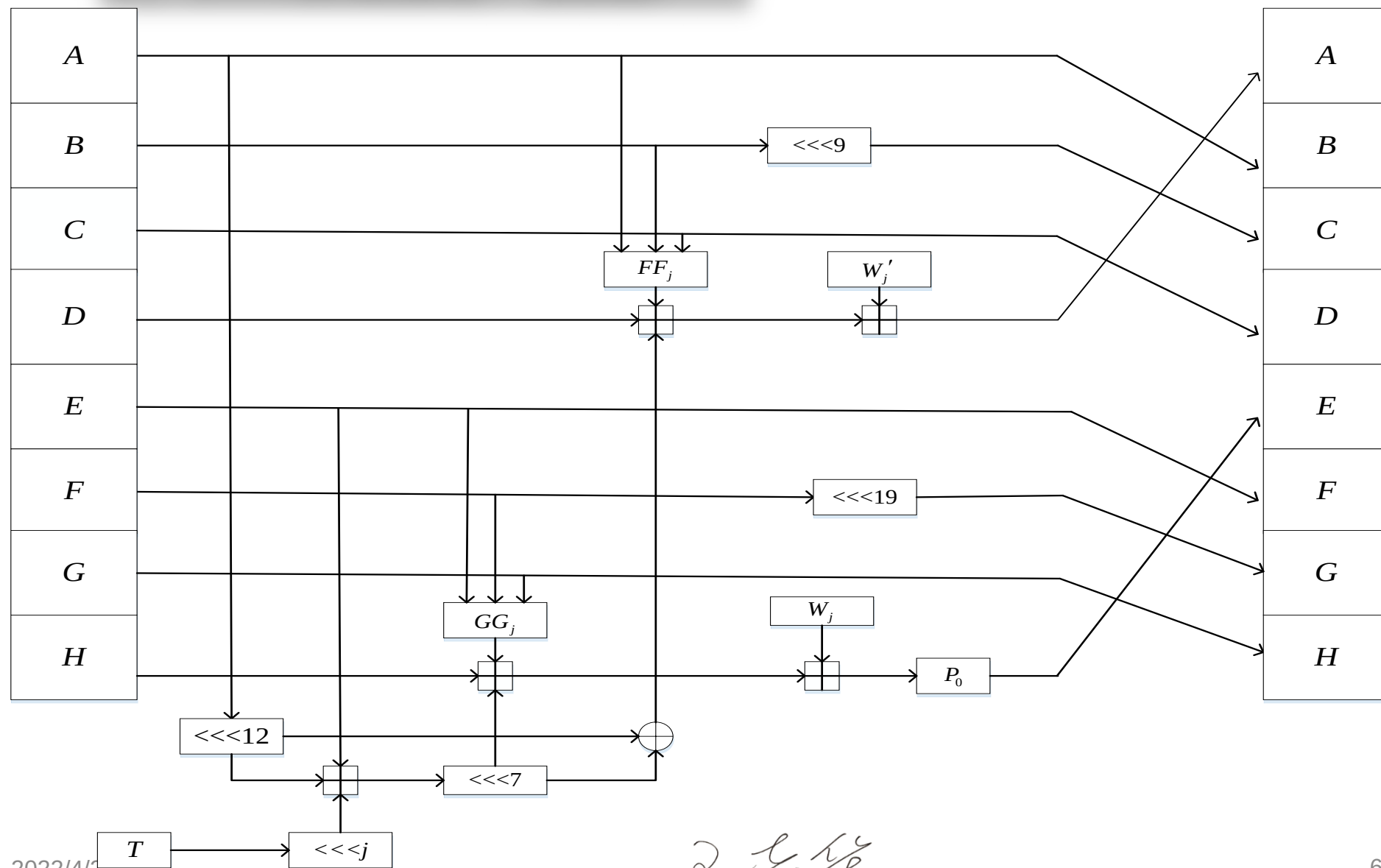
$$V^{(i+1)} = ABCDEFGH \oplus V^{(i)}$$

其中 $+$ 为模 2^{32} 加运算，字的存储为大端格式。图6-13是压缩函数中一步迭代示意图。

✓✓✓



3、SM3算法描述：第四步



2022/4/20



3、SM3算法描述：第五步

(5) 输出哈希值

$$ABCDEFGH = V^{(L)}$$

输出 256 比特的哈希值 $y = ABCDEFGH$ 。

2022



3、SM3算法描述：过程

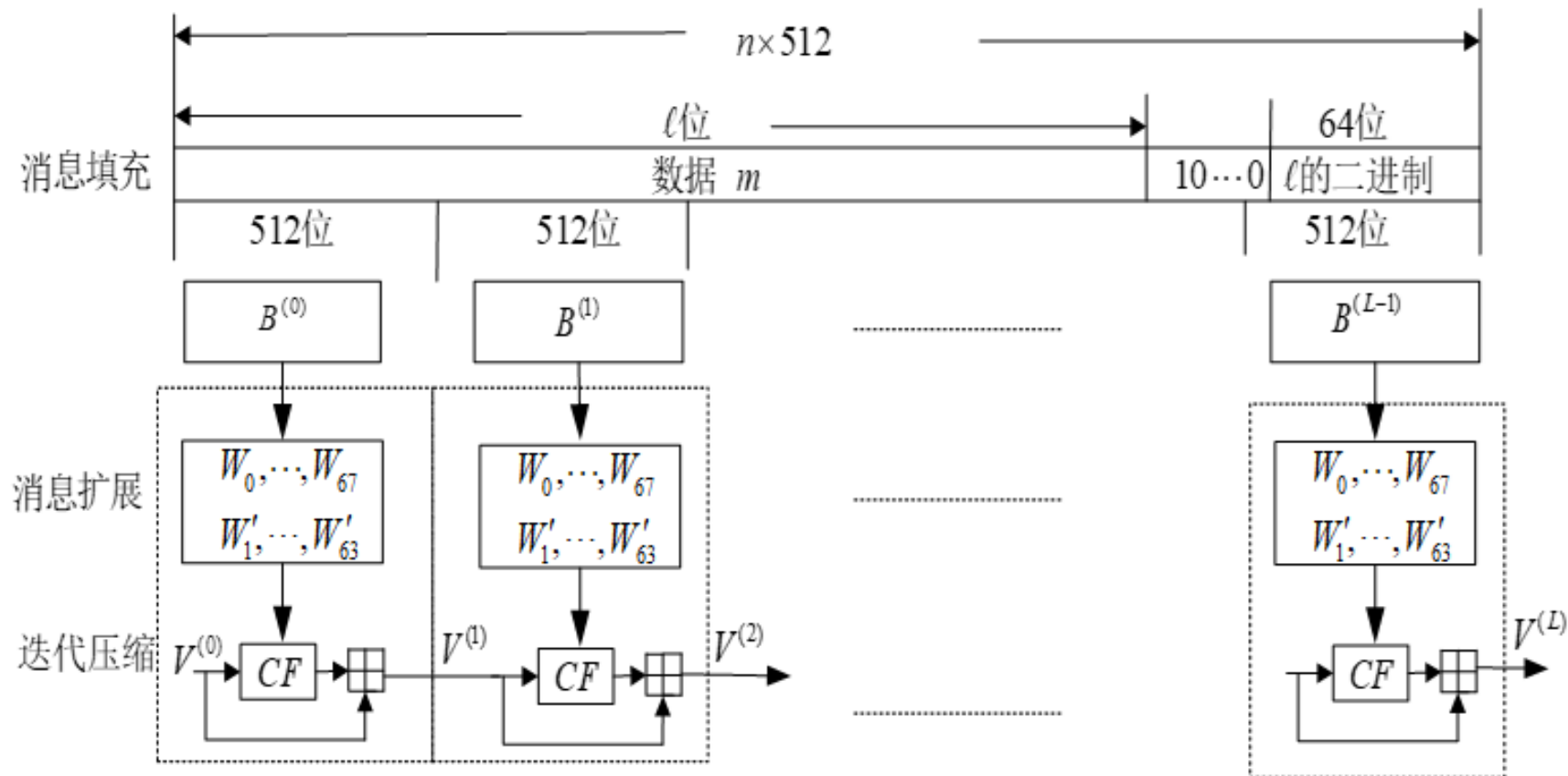


图6-14 SM3 产生消息哈希值的处理过程

2022/4/26



4、SM3安全性

压缩函数是哈希函数安全的关键，SM3的压缩函数 CF 中的布尔函数 $FF_j(X,Y,Z)$ 和 $GG_j(X,Y,Z)$ 是非线性函数，经过循环迭代后提供混淆作用。置换函数 $P_0(X)$ 和 $P_1(X)$ 是线性函数，经过循环迭代后提供扩散作用。再加上 CF 中的其他运算的共同作用，压缩函数 CF 具有很高的安全性，从而确保SM3具有很高的安全性。

2022/4/26

第4单元 哈希函数与消息认证码

提纲

CONTENTS

4.1 哈希函数概述

4.2 MD算法

4.3 SHA算法

4.4 MAC

4.5 SM3

4.6 其他哈希函数

王华

4.6 其他哈希函数



4.6.1 其他哈希函数概述

- 使用对称分组码构造Hash函数。
- 使用公钥算法构造Hash函数

2022/4/26

1、使用对称分组码构造哈希函数

- 特点：Hash值等于分组长度
 - $H_0 = IV$, IV是一个初始值.
 - $H_i = E_A(B) \text{ xor } C$

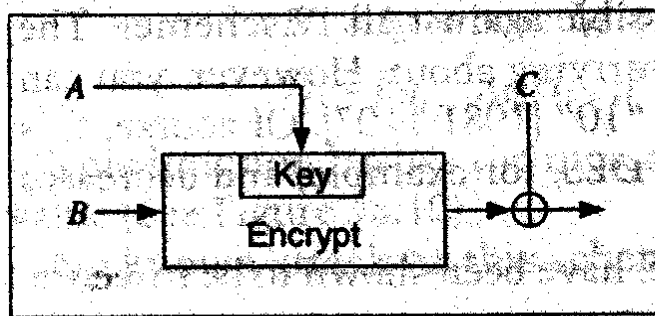
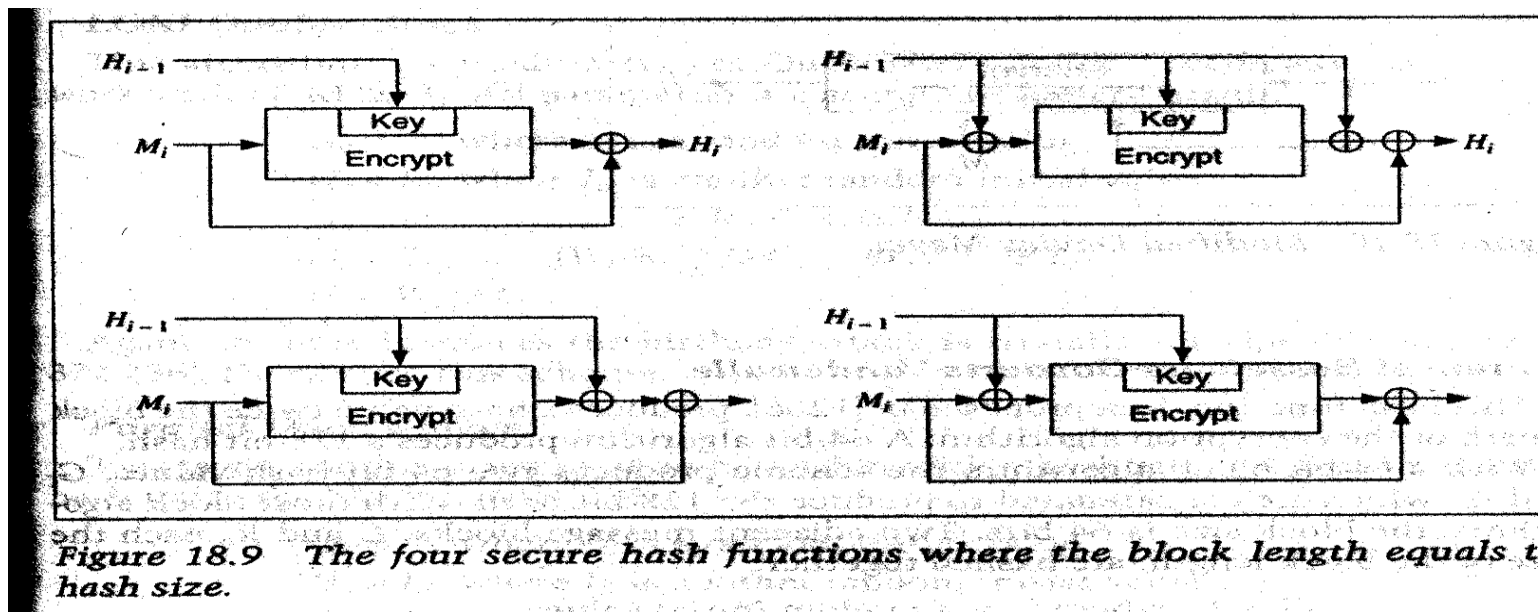


Figure 18.8 General hash function where the hash length equals the block size.

2024

1、使用对称分组码构造哈希函数



- $H_i = E_{M_i}(H_{i-1})$
- $H_i = E_{M_i \text{ xor } H_{i-1}}(H_{i-1})$
- $H_i = E_c(M_i \text{ xor } H_{i-1}) \text{ xor } M_i \text{ xor } H_{i-1}$

2024

1、使用对称分组码构造哈希函数

- $H_0 = I_H$, where I_H is random initial value.
- $I_H = E_{H_{i-1}, M_i}(H_{i-1})$

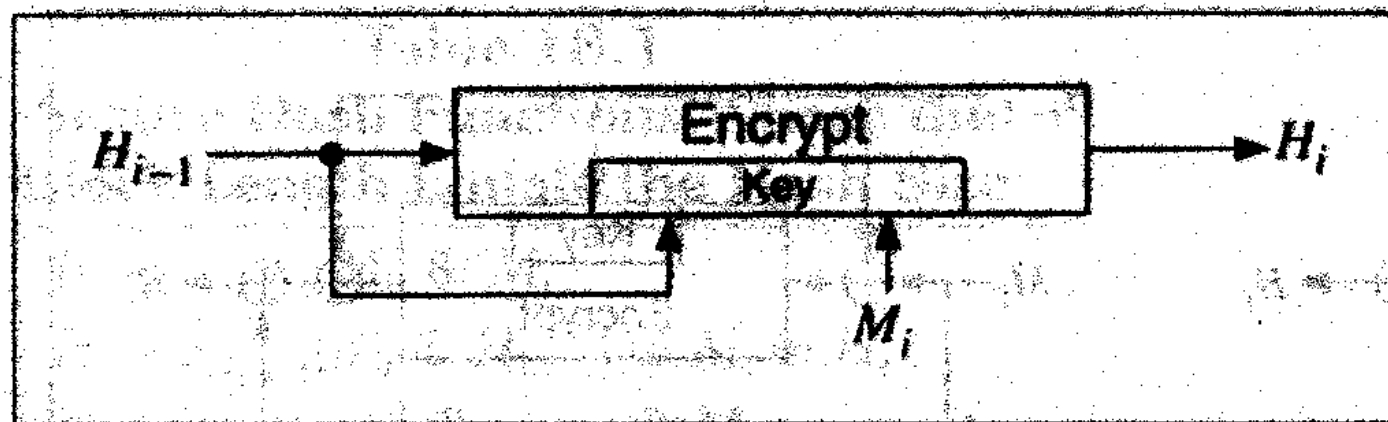


Figure 18.10 Modified Davies-Meyer.

2022/4/26

1、使用对称分组码构造哈希函数

- $G_0 = I_G$, where I_G is a random initial value
- $H_0 = I_H$, where I_H is another random initial value
- $W_i = E_{G_{i-1}, M_i}(H_{i-1})$
- $G_i = G_{i-1} \text{ xor } E_{M_i, W_i}(G_{i-1})$
- $H_i = W_i \text{ xor } H_{i-1}$

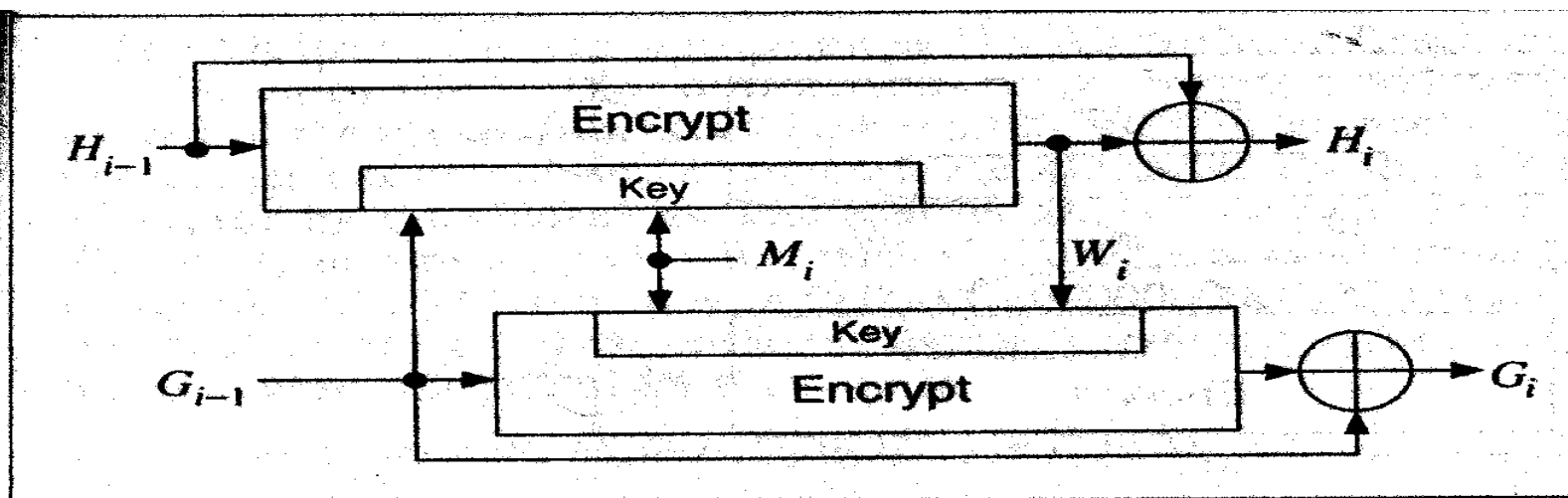


Figure 18.11 Tandem Davies-Meyer.

1、使用对称分组码构造哈希函数

- $G_0 = I_G$, where I_G is a random initial value
- $H_0 = I_H$, where I_H is another random initial value
- $G_i = G_{i-1} \text{ xor } E_{M_i, H_{i-1}}(\neg G_{i-1})$
- $H_i = H_{i-1} \text{ xor } E_{G_{i-1}, M_i}(\neg H_{i-1})$

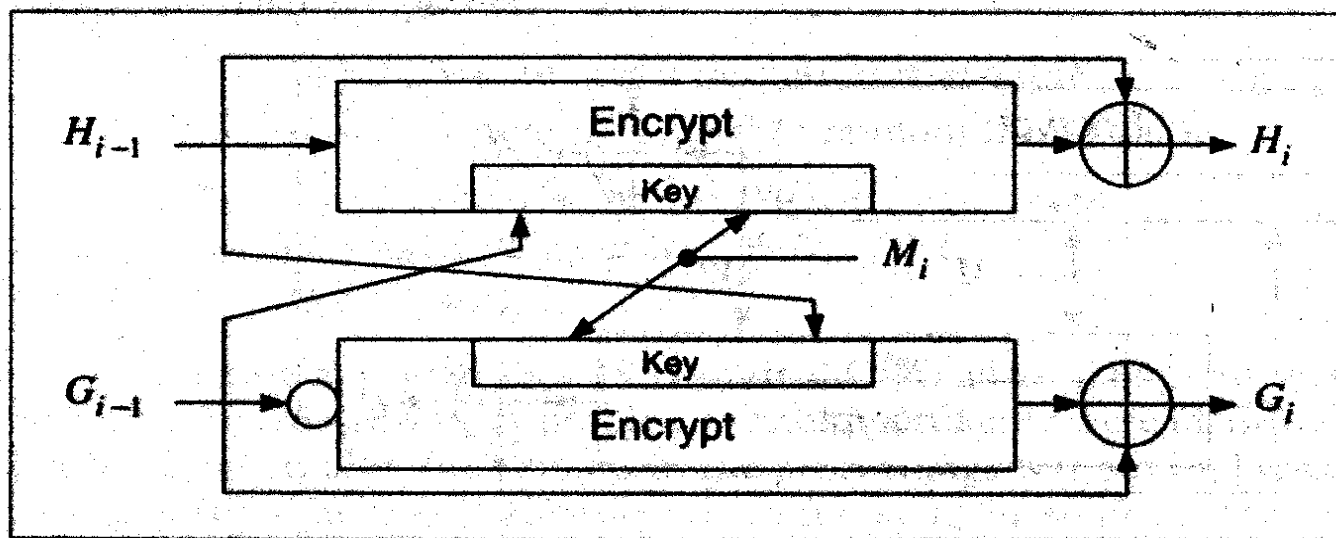


Figure 18.12 Abreast Davies-Meyer.



2、使用公钥算法构造哈希函数

- 例：用RSA构造

- $H(M) = M^e \bmod n$

- $H(M) = M^e \bmod p$

- 安全性

- 合适选择参数后，攻击与解离散对数一样困难。

- 弱点

- 速度很慢

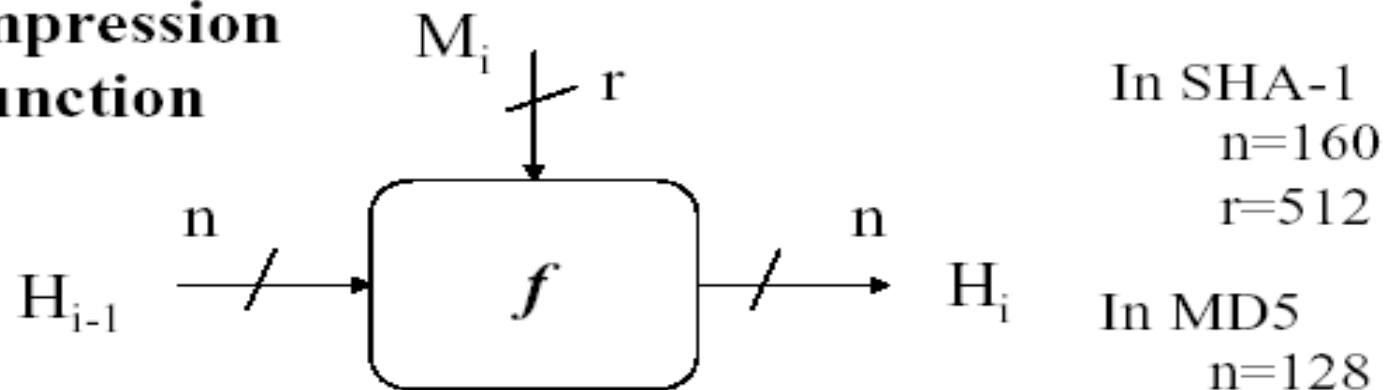
王华



3、构造哈希函数的基本方法：总结

General scheme for constructing a secure hash function

Compression
function



In SHA-1
 $n=160$
 $r=512$

In MD5
 $n=128$
 $r=512$

Entire hash

$$H_0 = IV$$

$$H_i = f(H_{i-1}, M_i)$$

$$h(m) = g(H_t)$$

2022/4/26

4.6 其他哈希函数



4.6.2 使用哈希函数

- 很多场景

2022/4/26



在线投标

- 假设Alice, Bob和Charlie都是竞标者
- Alice准备出价A, Bob准备出价B, Charlie C
- 他们都不相信自己的出价能保持机密
- 解决办法?
 - Alice, Bob, Charlie分别提交出价的哈希值 $h(A)$, $h(B)$, $h(C)$
 - 所有的哈希值被公布在网上
 - 当三个哈希值都收到后可以公布原始出价
- 哈希值并不暴露原始出价(单向性)
- 一旦提交哈希值, 不能改变原始出价(碰撞性)



清理垃圾邮件

- 如何清理
- 在我接受一个邮件之前，我希望确认发件者是花费了一定“功夫”来发送的，如花费了CPU时钟周期来创建邮件
- 只能发送有限的电子邮件
- 创建垃圾邮件的“成本”更高



清理垃圾邮件

- 设 M = 电子邮件
- 设 R = 要确定的值
- 设 T = 当前时间
- 发件者必须找到合适的 R 满足
 - 哈希(M, R, T) = (00...0, X), 并且
 - 哈希值的初始 N 位均为 0
- 发件者然后发送(M, R, T)
- 如果收件者能确认下面的条件, 则接受
 - 哈希(M, R, T)的前 N 位均为 0



清理垃圾邮件

- 发件者: 哈希(M,R,T)以N位0开始
- 收件者: 确认哈希(M,R,T)以N位0开始
- 发件者的工作量: 大约 2^N 哈希
- 收件者的工作量: 1次哈希
- 发件者的工作量以N的指数级增长
- 对收件者来说工作量保持不变
- 选择合适大小的N满足
 - 对普通用户来说可以接受
 - 对垃圾邮件发送者是难以接受的!